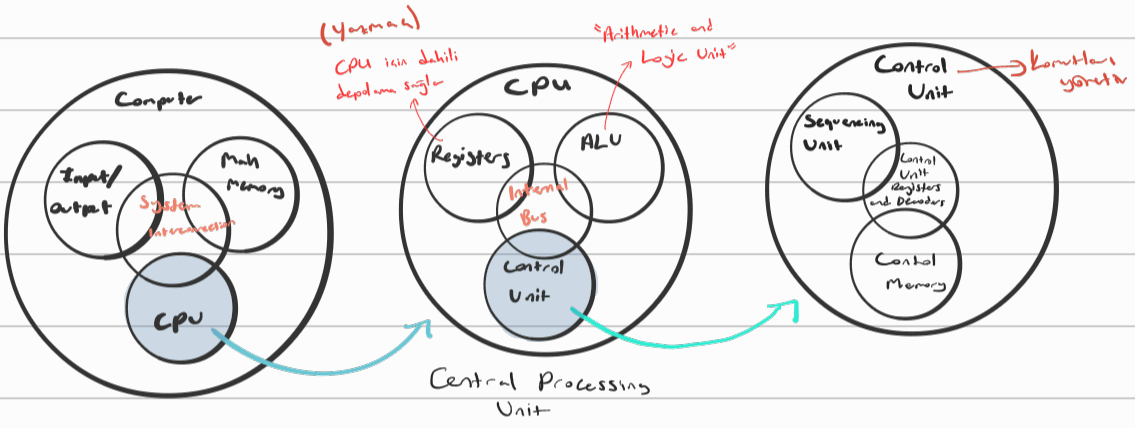
William Stallings, Computer Organization and Architecture 9th edition

Architectural (Mimari) tasarım → Programcının gördüğü ve kullanabildiği özellikler.
Sistem nitelikleri, komut setleri, bit sayısı, I/O mekanizmaları, veri tipleri (int, float)

Organizasyonel tasarım → Bilgisayarın donanım kısmı, mekanizması. Bu mimarinin donanımda nasıl gerçekleştirildiği. ALU tasarımı, pipeline yapısı, bus sistemi, clock hızı

Bir bilgisayarın gerçekleştirebileceği 4 temel işlem vardır:
- Veri işleme
- Veri depolama (Short-term, Long term)
- Veri Transferi (Input/Output: Dışarıdan bilgisayara bağlı bir cihazdan veri alınması/gönderilmesi)
  (Veri iletişimi: Daha uzaktaki bir cihaz ile)
- Kontrol



* En hızlı depolama yapabileceğimiz yer CPU'daki Registerlar.

* CPU'nun sekizdeklerinin her biri ayrı bir CPU gibi düşünülebilir. Her biri bir işlem birimidir. 4 sekizdeki mesela 4 tane CPU gibi. Multicore Processor denir bu çok sekizdeklilere. Processor da bu çok sekizdeki insanın silikon yapılmış denir.

Cache Memory (Ön Bellek) Processorun içinde bulunur Core'larla beraber

Ana harici bellek, CPU registor'a göre çok yavaştır. Bu önbellek de bu ikisinin arasındadır.

Bilgisayarın belleği aslında çok katmanlıdır ve ön bellek de bunlardan biridir.

* örneğin bir for döngüsü çalışırken içindeki satırlar cache'te tutulur. Yakın bir zamanda kullanacağı şeyleri tutar.

```
for{
==========
}
```

Ana bellekte tutsaydı bu çok yavaş olacaktı.

* Sekizdeğe en yakın Seviye 1 (L1) dir ve L2, L3 diye gider. İndis arttıkça boyutu artar ama hızı yavaşlar.

Processor içinde L3 cacheler bulunur mesela. Oradaki Core'ların içinde de L1-L2 bulunur.

* L1-I cache (I → instruction) ve L1 data cache var bir Core'un içinde mesela.

L1-I: işlemcinin çalıştıracağı makine komutlarını saklar. Fetch (getirme) sırasında CPU önce L1-I, sonra L2, L3 ve en son RAM'i kontrol eder.

L1 data cache: programın kullandığı verileri saklar. int a = b + c; gibi değişken değerlerini burada saklar. CPU, execute aşamasında veriyi buradan alır.

L3 önbellekler tüm Core'lar tarafından ortak kullanılır. L1 ve L2'ler ise Core'a özel

L1-I ve L1-D bunlar mümkün değilse core L2'yi kullanır. Daha hızlı daha düşük 2'ye bölmüşler.

L2 data + instruction. Birlikte yapar bunları. L1 gibi ikiye ayrılmamıştır. bu da dolunca L3'ü kullanır.

L3

* L1'in ikiye bölünmesinin amacını şöyle gibi düşün: Bir araba fabrikasında tek tek bir arabayı üretmiyorlar. Hepsinin ilk şasisini, motorunu, kaportasını vs. yapıp hemen hemen aynı anda bir sürü araba bitiriyorlar. Bu da onun gibi, Pipeline demiş buna da. CPU'nun boş kalmasını da engeller.

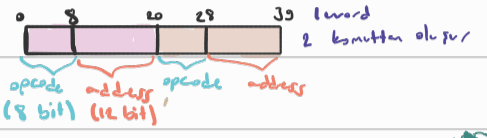
CPU → L1 → L2 → L3 → RAM → Disk (veri hiyerarşisi)



IAS Computer: Programı depolama üzerine bir dizayn felsefesi

Önceden vakum tüpleri kullanılıyordu (1. nesil bilgisayarlar) sonra transistör (2. Nesil bilgisayar)

IAS bellek yapısı word (kelime) denen parçalardan oluşur



opcode (operation code) → ne yapılacağını söyler

address → hangi bellek adresiyle çalışacağı.





Registers

Memory Buffer Register (MBR): Bellekten gelen/gidecek veriyi tutar

Memory Address Register (MAR): Hangi bellek adresine erişmek istediğini tutar.

Instruction Register (IR): İcra edilen 8 bitlik opcode komutunu içerir.

// Buffer Register (IBR): Aynı word içindeki 2. komutu geçici olarak tutar

Program Counter (PC): Bellekten gelecek bir sonraki komut çiftinin adresini tutar.

Accumulator (AC) ve Multiplier Quotient (MQ): ALU operasyonlarının operandlarını ve sonuçlarını geçici olarak tutar.

Integrated Circuits (3. nesil bilgisayarlar) çipler kullanılmaya başlandı, yüzlerce transistör yani

cache, pipeline, superscalar bilumum bunların hepsi ancak 3. nesilden sonra mümkün oldu.

IBM Bilgisayar: Şu anda'daki bilgisayardan, International Bussiness Machine

Mimarileri aynıdır, organizasyonları (donanımları) güncellenip geliştirilir sadece

Ayrık Bileşen: Tek bir bağımsız transistör (2. nesil bilgisayarlarda kullanılanlar) Üretimi pahalı ve zahmetli

Moore Yasası: Bir chip içine sığan transistör sayısı her yıl 2 katına çıkabilmiş diye tahmin etmiş Moore

Bus yapısı → Veri yolu. CPU, main memory, I/O ile arasında kurulan iletişim yolu gibi düşün.

Magnetic Core Memory (Eski): Pahalı, hantal ve destructive teknoji kullanıyordu. Veri okunduktan sonra siliniyordu.

Microprocessor (Yeni): Tek bir yongaya daha fazla öğe yerleştirildi

ARM ve Intel işlemciler: Intel geçmiş bilgisayarlarda kullanılırken, ARM mobil cihazlarda falan kullanılır.

ARM, daha az enerji harcar ve ısınmaz.

Deeply Embedded Systems: Cihazda program ROM'a yazıldığından bir daha programlanmaz

(read only memory)

3. CPU'nun İç Yapısı

CPU dediğimiz yonganın içinde şu birimler vardır:

- **ALU (Arithmetic Logic Unit)** → matematik işlemleri
- **Register** → çok hızlı küçük hafıza
- **Control Unit** → komutları yönetir
- **Cache** → hızlı veri depolama

Yani:

```
CPU (bir yonga)
  ├ ALU
  ├ Register
  ├ Control Unit
  └ Cache
```

👉 **Önemli:**
Bu saydıkların **yonganın içinde bulunan alt birimlerdir**, ayrı fiziksel parçalar değil.

Microprocessor Speed

Pipelining: Araba fabrikasında her arabanın bir aşamasını beraber yapmaları.

1 Branch Prediction: Her if ve loop komutlarına branch (dallanma) denir. CPU pipeline çalışırken bu if'lere takıldığında çok yavaşlar. Bunu düzeltmek için de if'lerin sonucunu tahmin eder. "True bu sefer" der mesela ve doğrusuna çok hızlı bir şekilde sırasıyla ilerler.

2 Speculative Execution: Branch prediction ile birlikte çalışır. Sadece tahmin etmekle kalmaz, gerçekleştirir.

3 Data flow Analysis: Hangi komutlar aynı anda çalışabilir onu tespit ediyor. Mesela $\begin{matrix} a=b+c \\ d=e+f \end{matrix}\rangle$ bu bağımsız komutları paralel çalıştırır.

4 Superscalar Execution: Aynı anda CPU birden fazla işlemi çalıştırır. İçinde ALU1 ve ALU2 var mesela her biri farklı bir toplama işlemi yapıyo

İşlemci, tek bir saat (clock) döngüsünde birden fazla bağımsız komutu çalıştırabilir.

★ Bilgisayarın diğer bileşenleri, CPU'nun hızına yetişememeye başladılar ve ardından büyük bir performans farkı oldu. Bu farkı dengelemek için mimariyi ayarlamaya çalıştılar:

- DRAM'ler "daha derin" yerine "daha geniş" yapılar böylece tek seferde gelen bit sayısı artabilir.
- İşlemci ve ana bellek arasına önbellek eklenmiş ki bellek erişim sıklığı azalsın.
- DRAM arayüzünü değiştirmişler. Önbellek veya başka bi önbelleğe alım mekanizması eklemişler.
- Daha yüksek hızlı veri yolları ve veri hiyerarşisi ile işlemci—bellek arası bant genişliği artırılmış.

Bir süre sonra artık önbellekleri artırmak da yetmiyor. Ve farklı bir yaklaşıma gidecek Multicore teknolojisine geçiyorlar. Birden fazla işlemci, büyük ve paylaşılan bir önbellekle aynı yongaya konuluyor.

Böylece:

- Saat hızı artırılmadan performans artırılabilir.
- Çok karmaşık tek bir işlemci yerine daha büyük 2 işlemci kullanılır.
- İki işlemci ile daha büyük önbellekler mümkün olur.
- Önbellekler büyüdükçe bir yonga üzerinde 2-3 seviyede önbellek oluşturmak performans açısından anlamlı hale geldi.

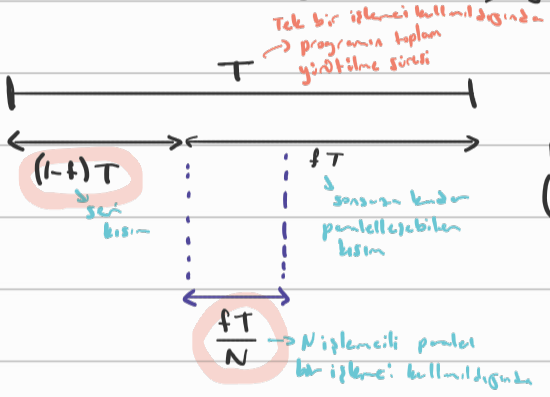
MIC (Many Integrated Core): Tek çip üzerinde birden çok aynı tip CPU çekirdekleri bulunur. Paralellik ve performans artışı. Ama kötü yanı: Bir kadar çok çekirdek kullanmak gösterişten öte.

GPU (Graphics Processing Unit): Başta grafik işlemleri (2D/3D) için tasarlanmış, ama asıl gücü çok sayıda veriyi aynı anda işleyebilmesi. Tekrar eden hesaplamalar, büyük veri setleri, aynı işlemi birden fazla veri üzerinde aynı anda yapar (vector processing)

Artık genel amaçlı hesaplama için de kullanılmaya başlanmış (GPGPU)

Amdahl Kanunu

Bir işi sonsuza kadar hızlandıramazsın. Bi hızlandırılabilen paralel kısmı, bi de ne yaparsan yap hızlandıramayacağın seri kısmı vardır. O yüzden seri ne kadar çekirdek kullanırsan kullan, seri kısım hep seni yavaşlatacak. Sonsuza kadar hızlanamazsın.



$$\text{Max hız artışı} \left(\frac{T}{\text{processor ile den zaman}}\right) = \frac{1}{(1-f) + \frac{f}{N}}$$

Örn Bir program %80'i paralel %20'si seri. 4 çekirdekte çalıştırılıyor. Speedup?

$$\frac{1}{0{,}2 + \frac{0{,}8}{4}} = \frac{1}{0{,}4} = \boxed{2{,}5}$$

Örn %10 seri kısım olsa ve çekirdek sayısı sonsuz olsa max speedup kaç olur?

$$\frac{1}{0{,}1 + \frac{0{,}9}{\infty}} = \boxed{10}$$

Little's Law

$L = \lambda \cdot W$

- L → sistemdeki ortalama iş sayısı
- λ → birim zamanda gelen iş sayısı
- W → bir işin sistemde geçirdiği ort. süre

Little Law'ın geçerli olması için sistem stable (kararlı) olmalı. Yani: Gelen iş sayısı ≤ işlenen iş sayısı (yoksa kuyruk sonsuza çıkar.)

Performans Faktörleri

I_c: Komut (instruction) sayısı

p: Bir komut için gereken işlemci döngüsü sayısı (CPI)

m: Bellek erişim sayısı

k: Bellek erişim gecikmesi / maliyeti

τ: Saat çevrim süresi (Clock period, $\tau = 1/f$)

1- Instruction Set Architecture (ISA)

- I_c ve p etkiler.

Komut seti ne kadar karmaşıksa, aynı iş: daha az komutla yapılır → I_c azalır

2- Compiler Technology

- I_c, p ve m etkiler

İyi optimize bi derleyici:

Daha az komut üretir → I_c ↓

Daha verimli komut dizisi → p ↓

Bellek erişimlerini azaltabilir → m ↓

3- Processor Implementation (işlemci gerçeklenmesi)

- p ve τ etkiler

Pipeline, superscalar gibi teknikler, CPI'yi düşürür → p ↓

Donanım tasarımı saat hızını belirler → τ

4- Cache and Memory Hierarchy

- k ve τ etkiler

Cache kullanımı, bellek erişim maliyetini düşürür → k ↓

Bellek sistemi, saat süresini de etkileyebilir → τ

CPI_i, komut tipi i için gerekli döngü sayısı olsun. I_i de programda çalıştırılan komut sayısı

$$\text{genel } CPI = \frac{\sum_{i=1}^{n} (CPI_i \times I_i)}{I_c}$$

$$T = I_c \times \underbrace{CPI}_{p + (m \times k)} \times \tau$$

T: programı yürütmek için gereken işlemci süresi

MIPS rate: Bir işlemci için genel bir performans ölçüsü (Million Instructions per Second)

$$MIPS = \frac{I_c}{T \times 10^6}$$

→ T'de yerine koyduğumuz zaman:

$$MIPS = \frac{f}{CPI \times 10^6}$$

Örn

10 milyon Instruction çalıştırılıyor

CPI = 2

Clock Rate = 2 GHz ise Execution Time nedir?

Ana formül: Execution Time = I_c × CPI / Clock Rate

$= 10.10^6 \cdot 2 \;/\; 2.10^9$

$= 10.10^{-3} =$ **0,01 saniye**

Örn

Bir CPU'nun Clock Rate = 1 GHz, CPI = 2 ise MIPS nedir?

MIPS = Clock / (CPI × 10^6)

$= 10^9 \;/\; 2 \times 10^6 =$ **500 MIPS**

Arithmetic Mean (AM): Ölçtüğün şey zaman (execution time) ise AM kullanılır.

Geometric Mean (GM): oranlar / speedup / performans karşılaştırması varsa GM kullanılır.

* **Benchmark** = Karşılaştırma , **SPEC** = performans benchmark seti. Bilgisayarı, birden fazla kodda test ediyor.

SPEC Temel Terimleri:

1) **Benchmark:** Performansı ölçmek için kullanılan test programı. Yüksek seviyeli dilde yazılır (C, C++ gibi)
2) **System Under Test (SUT):** Test ettiğin bilgisayar
3) **Reference Machine:** Karşılaştırma için kullanılan standart sistem. Referans süre elde edilir.
4) **Base Metric:** Standart ve dürüst ölçüm, herkes aynı kurallarla ölçer ki adil olsun.
5) **Peak Metric:** Maksimum performansı görmek için yapılan ölçüm, sistem en iyi hale getirilir.
6) **Speed Metric:** Tek bir işi ne kadar hızlı bitirdiğini ölçer
7) **Rate Metric:** Aynı anda kaç iş yapabildiğini ölçer.

Von Neumann Mimarisinin 3 temel kuralı:

1) Veriler ve komutlar RAM'de aynı yerde bulunur
2) Her şeye adresle erişilir
3) Komutlar sırayla çalışır

Hardwired Program: Donanımın kendisi aslında program. Mesela normalde bir program yazarsın ve CPU onu çalıştırır. Ama hardwired'da direk mesela toplama devresi kuruyorsun ve o toplama işini yapar. Özel amaçlıdır. Değiştirmesi zordur. Kullanımı özel amaçlıdır.

Interrupt (Kesme) Türleri:

1) **Program Interrupt:** Programın içinde hata durum (Sıfıra bölme, overflow vb.)
2) **Timer Interrupt:** CPU içindeki zamanlayıcıyı tetikler
3) **I/O** =: Disk işlemi bitti, klavyeden tuş okundu gibi CPU'ya haber verir
4) **Hardware Failure:** Elektrik kesintisi, bellek hatası

* Eğer interrupt'lar olmasa CPU boş boş beklerdi. Interrupt gelince CPU o işe geri bakıyor.

Sequential Interrupt Processing (Sıralı Kesme işleme): Bi interrupt geldiğinde diğer interruptlar devre dışı bırakılır. Basit ve karışıklık yoktur. Fakat Dezavantajı: Acil bi interrupt beklemek zorunda kalabilir.

Nested Interrupt Processing (iç-içe Kesme işleme): Kesmeler için priorities (öncelikler) tanımlanır. Yüksek öncelikli interruptlar önce işlenir.

Direct Memory Access (DMA): I/O Modülünün, işlemcinin veri alışverişi (exchange) sorumluluğunu ortadan kaldırarak belleğe okuma/yazma komutları vermesidir.

Bus Interconnection: Aynı zaman diliminde iki cihaz iletişim/aktarım yaparsa sinyalleri üst üste biner ve bozulur.

System bus: Başlıca bilg. bileşenlerini (CPU, bellek, I/O) bağlayan bir veri yolu.

1- Data Bus: Veri yoludur, veriyi taşır. Çift yönlüdür. CPU ↔ RAM ↔ I/O

2- Address Bus: Verinin nereden alınıp nereye yazılacağını söyler

3- Control Bus: Veri ve adres hatlarını kontrol eder.

★ Bir modül diğerine veri göndermek istiyorsa: 1) Veri yolu kullanımını elde etmelidir. 2) Veri yolu üzerinden veri aktarmalıdır.

★ Bir modül başka bir modülden veri talep etmek isterse: 1) Veri yolu ////. 2) Uygun kontrol ve adres hatları üzerinden diğer modüle talep aktarmalıdır. Daha sonra 2. modül veri göndermeyi bekler.

Point-to-point Interconnect (PPI)

Shared bus yapısı yetersiz gelmeye başladı. Trafik sıkışıklığından. Onun yerine her bileşen doğrudan birbirine bağlandı, özel bağlantılar yapıldı.

QPI (Quickpath Interconnect): Yüksek hızlı PPI bağlantılarıdır. CPU ↔ CPU veya CPU ↔ I/O

Küçük paketler halinde aktarır ve bu paketlere flit (flow control unit) denir.

Cache (Önbellek)

Cache ↔ işlemci → word transfer

Cache ↔ Ana bellek → block transfer

Sonda

★ D/Y, bağlık doldurma her durumda tam sayı olacak

Bellek Sistemleri (Sınıflandırması):

1) **Location:** Dahili/Internal (Register, önbellek, ana bellek) ve Harici/External (Sabit diskler, optik diskler (CD/DVD) ve manyetik şeritler/teyp)

2) **Kapasite:** Word (kelime) sayısı veya Byte cinsinden ifade edilir.

3) **Aktarım Birimi:** Dahili bellek için: Word (veri yolu bit genişliği), Harici b. için: Block (çok daha büyük)

4) **Erişim Yöntemi (Access Method):**

- **Sequential access:** Sıralı erişim. A→B→C→D→E örnek: Manyetik teyp
- **Direct access:** İstediğin yere yaklaşık olarak atlayabiliyorsun. Doğru bölgeye gidip arıyor. örnek: HDD
- **Random access:** Herhangi bir noktaya doğrudan ve aynı sürede erişilir. Erişim süresi sabit. Örnek: RAM
- **Associative access:** Veriyi adresle değil içeriğine göre bulur. Tüm bellek konumları aynı anda taranır. Çok hızlı ama çok pahalı. örnek: Cache

Hız: Associative > Random > Direct > Sequential

5) **Performans:** Belleğin performansı için 3 parametre var:

Access time (Latency): Random erişimli bellek için, okuma-yazma işlemleri için gereken süredir. Random erişim olmayan belleklerde için, okuma-yazma mekanizmasını istenen konuma getirmek için gereken süredir.

Memory Cycle Time: Erişim süresi + 2. erişimin başlayabilmesi için gereken ek süre. İşlemci ile değil, sistem busu ile ilgili

Transfer rate: Veriden bir bellek birimine veya diğerine aktarılabilme hızı. Random bellekler için → $1/\text{cycle time}$

6) **Fiziksel Tür:** Yarı iletken, Manyetik, Optik, Manyeto-optik.

7) **Fiziksel Özellikler:**

- **Volatile (Geçici/Uçucu) bellek:** Elektrik gittiğinde kaybolur. (Örn: RAM)
- **Nonvolatile (Kalıcı) bellek:** Bilgileri tutmak için elektrik gücüne gerek yoktur. (Örn: ROM, Disk)
- **Magnetic Surface Memories:** Kalıcıdır (Nonvolatile)
- **Yarı iletken bellek (Semiconductor):** Geçici veya kalıcı olabilir.
- **Silinemez bellek:** Salt okunur bellek (ROM), tahrip edilmediği sürece değiştirilemez.



Bellek Hiyerarşisi: Kapasite, erişim sayısı ve Maliyet arasında bir denge vardır.



- Bit başına maaliyet azalır.
- Kapasite artar
- Erişim sayısı artar (yavaşlar)
- İşlemci (CPU) tarafından erişim sıklığı azalır.

- Önbellek, ana bellekle CPU arasına yerleştirilen, küçük ve hızlı, SRAM tabanlı bir yapıdır.
- Ana bellek, her biri K kelimeden oluşan M adet bloktan meydana gelir. Buna göre toplam kelime sayısı:

$N = M \times K$ 2^n adres üretir.

- Önbellek ise C adet Satır (Line) veya Slot içerir. Her bir satır, ana bellekteki bir bloğun boyutuna (K kelime) tam olarak eşittir.
- Önbellekte, Satırdaki verinin ana bellekteki hangi bloğa ait olduğunu ayırt etmek için bir etiket (Tag) alanı ve verinin geçerli dolu olmadığını gösteren bir Valid Bit bulunur.
- $C < M$ (Önbellek satır sayısı, ana bellek blok sayısından çok küçüktür)

Önbellek Okuma Operasyonu

1-) CPU bir adres üretir 2-) Bu adresin önbellekte olup olmadığı kontrol edilir (Cache-hit/miss)

3-) Hit olursa veri doğrudan çok hızlı bir şekilde önbellekten CPU'ya iletilir.

Miss olursa istenen veriyi içeren tüm blok ana bellekten okunur ve önbellekteki satırlardan birine yerleşir. (gerekirse eski bloğu siler hatta) ve eş zamanlı olarak CPU'ya teslim edilir.

Önbellek büyüdükçe hit oranı artar ancak arama hızı yavaşlar ve çip üzerinde kapladığı alan (maaliyet) devasa boyutlara ulaşır.

Mapping Functions

Ana bellekteki blokların önbellekteki satırlara nasıl yerleşeceğini belirleyen 3 temel yöntem vardır:

(S = Ana bellek blok adresi bit sayısı , W = Kelime/Byte seçimi bit sayısı)

1-) Direct Mapping (Doğrudan Eşleme) L1 cache

En basit teknik. Her ana bellek bloğunu yalnızca bir önbellek satırına (line) yerleşebilir.

$i = j \pmod C$ (i = önbellek satır numarası, j = Ana bellek blok numarası, C: önbellekteki toplam satır sayısı)

CPU'nun ürettiği adres J'e bölünür:

Tag (etiket)	Line (Satır)	Word (Kelime)

Word → Blok içindeki kelimeyi seçer

Line → Hangi önbellek satırına bakılacağını söyler

Tag → O satırdaki bloğun gerçekten aranan blok olup olmadığını doğrular.

Dezavantajı: Sürekli yer Değiştirme Problemi (Thrashing). Eğer bir program aynı satıra maplenen iki farklı blok'taki veriye ardışık olmak çağrılırsa (örn: j=0 ve i=C), önbellek sürekli miss eder ve blokları durmadan birbirinin üzerine yazar. Hit oranı düşer.

Victim Cache: Istenilen cezasını (miss penalty) düşürmek için bir yöntemdir. Daha sonra tekrar sorulabilir diye neyin atıldığını hatırlar. Böylece küçük bir maliyetle tekrar kullanılabilir. Geri dönüşüm gibi yani.

2-) Associative Mapping L1 ile bir sonraki bellek düzeyi arasında bulunur.

Herhangi bir ana bellek bloğu önbelleğin herhangi bir satırına yerleşebilir. Boş alan veya silinmesi uygun görülen herhangi bir slot kullanılabilir.

Tag (Etiket)	Word (Kelime)

Önbellekte spesifik bir satır adresi olmadığı için, CPU adresteki Tag kısmını alır ve önbellekteki tüm satırların etiketleriyle aynı anda (paralel olarak) karşılaştırılır.

Dezavantajı: Bu yöntem ilişkisel bellek donanımı gerektirdiği için pahalıdır. Büyük boyutlarda ve devre tasarımı pahalıdır.

3-) Set-Associative Mapping modern işlemcilerin kullandığı tasarım (L1, L2, L3)

İkisinin avantajlarını birleştiren hibrit yöntemdir. Önbellek, kendi içinde kümelere (sets) ayrılır.

Her kümenin içinde k adet satır bulunur. Buna k-way set associative denir. (Örn: 2-way, 4-way, 8-way)

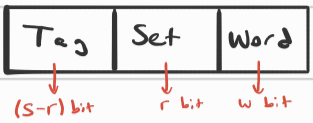
$v = j \pmod Q$ (v: küme numarası, j: Blok numarası, Q: Toplam küme sayısı) $Q = C/k$

Bir blok hangi kümeye gideceğini doğrudan bilir (Direct), ama o kümenin içindeki k adet satırdan istediğine yerleşir (Associative)

Tag (etiket)	Line (Satır)	Word (Kelime)

Adresteki Set bilgileri ile ilgili kümeye yazılır. O kümenin içindeki k adet satırın etiketi paralel olarak tanır.

Direct Mapping formülleri:



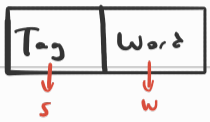
$(s+w)$ bit → adres uzunluğu

2^w words/bytes → blok veya satır boyutu

$\frac{2^{s+w}}{2^w} = 2^s$ → ana bellekteki toplam blok sayısı

$m = 2^r$ → önbellekteki toplam satır sayısı

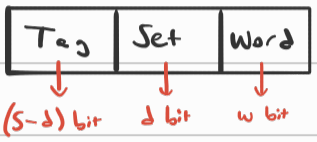
Associative Mapping formülleri:



2^w = Blok boyutu yani

s bitlerinin tamamı değişken Tag olarak kullanılır.

Set-Associative Mapping formülleri:



2^w blok boyutu yine.

$v = 2^d$ → önbellekteki toplam küme sayısı

1 Kilobayt = 2^{10} = 1024 byte

1 Megabayt = 2^{20} byte

1 Gigabyte = 2^{30} byte

⋮

RAM volatile

- DRAM (dinamik ram): Kondansatör kullanılır. Sürekli yenileme (refresh) ister. Yavaş ve ucuz. Bildiğimiz RAM'lerde kullanılır.
- SRAM (statik ram): Flip-flop devresinde veriyi saklar. Refresh gerektirmeden tutar. Güç kesmediği sürece ok. Çok hızlıdır. CPU önbelleğinde (L1, L2, L3) kullanılır. Daha pahalıdır.

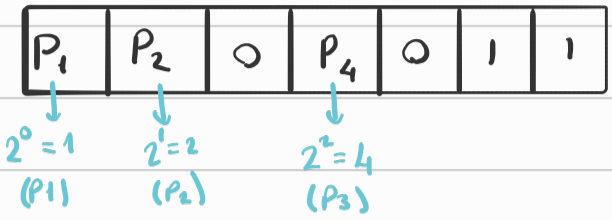
ROM (Read-only memory): Salt okunur bellek. Kalıcıdır (non-volatile). RAM'den daha yavaş. Türleri:

- ROM → fabrikada üretilirken içine yazılır, hiç değiştirilemez.
- PROM (Programmable ROM) → Kullanıcı tarafından 1 kez yazılabilir sonra değiştirilemez.
- EPROM (Erasable PROM) → Daha esnek PROM gibi. Silme işlemi birkaç kez yapılabilir.
- EEPROM (Electrically Erasable PROM) → Flaş belleğin atası. Önceki içerikleri silmeden üzerine bilgi yazabilir.
- Flash Memory → EPROM'la EEPROM arası bilgi. Bayt düzeyinde silme sağlamaz. Bellek hücrelerinin bir bölümü tek bir işlemle (flaşla) silinir. USB ve SSD'ler bu dur.

Hamming Code

Veri bitleri → benim gönderdiğim kod (0101 gibi)

Parity bitleri → bir hata olduğunda haber veren bitler. Her zaman 2'nin kuvveti olan pozisyonlara yerleşirler. Örneğin:



P'lerin kontrol ettikleri bitler:

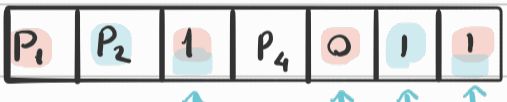
P_1 ⟶ 1 bit atla, 1 bit kontrol et... (1, 3, 5, 7 ....)

P_2 ⟶ 2 bit kontrol et, 2 bit atla... (2, 3, 6, 7, 10, 11 ....)

P_4 ⟶ 4 bit kontrol et, 4 bit atla... (4, 5, 6, 7, 12, 13, 14, 15 ....)

★ Parity bitleri, kontrol ettikleri bitlerdeki toplam 1 sayısı ÇİFT sayıda ise yerleşirler. Yani 0 olur (hata yoktur.) TEK ise 1 olur.

Örnek: 1011'lik bi veri gönderiliyor.



Verileri yerleştirdik.

P_1 = 1, 3, 5, 7. sıralara bakıyorduk. ⇒ ~~P1~~ 0 1 0 1 = 0 (Ekon'lara. Aynı olunca önemsiyodu) — bunu görmene gerek yok — çift sayıda

P_2 = 2, 3, 6, 7. sıralara bakar ⇒ 1 ⊕ 0 ⊕ 1 ⊕ 1 = 1 Durumu çift yapmak için P_2 kendini 1 yapar! — tek sayıda

P_4 = 4, 5, 6, 7 ⇒ 0 ⊕ 1 ⊕ 0 ⊕ 1 = 0

Paketin son hali (Hamming kodu): 0 1 1 0 0 1 1 — karşı taraf bu kodu alır. Ama yolda kayıp olur ve 5. bit bozulup 0 yerine 1 olursa mesela:

Sendrom (Decimal'e çevrildiğinde hatalı bitin sırasını verecek):

Yeniden Parity bitlerini sorgularız.

P_1 = 1, 3, 5, 7. sıralar ⇒ 0 ⊕ 1 ⊕ 0 ⊕ 1 = 1 Hatalı!

P_2 = 2, 3, 6, 7. ⇒ 1 ⊕ 1 ⊕ 0 ⊕ 1 = 0 temiz.

P_4 = 4, 5, 6, 7. ⇒ 0 ⊕ 0 ⊕ 1 ⊕ 1 = 1 Hatalı!

★ büyükten küçüğe doğru

Sendrom değeri (P_4 P_2 P_1): 1 0 1

$(101)_2$ = 5. bitte hata vardır.

RAID (Redundant Array of Independent Disks): Tek bir büyük ve hızlı disk üretmek yerine birden fazla ucuz disk yan yana bağlanarak performansı artırmadır.

RAID-0 (Striping / Şeritler halinde): Veriler bloklar halinde sırayla disklere dağıtılır. Bir disk bozulursa tüm veri gider. Performans maksimumdur. Tüm diskler aynı anda paralel çalışır, hızlıdır.

RAID-1 (Mirroring): Her diskin tam bir kopyası (aynası) yanındaki ikinci bir diske birebir yazılır. Kapasite verimliliği %50'dir (2 Tb veri için 4 Tb disk gerekir). Güvenilirlik çok yüksektir.

RAID-2 (Hamming Code ile düzeltme): Parity bitleri ayrı, ayrı özel disklerde saklanır. Dezavantajı çok fazla disk gerektirmesidir. Ticari olarak tercih edilmiyor.

RAID-3 (Bit düzeyinde eşlik): Tüm veri disklerinin bitleri XOR'lanarak tek bir Parity değeri üretilir ve bu değer tek bir özel parity diskinde saklanır.

RAID-4 (Blok düzeyinde eşlik): Raid 3 ile aynı, ama veri disklere bit değil blok düzeyinde dağıtılır. Dezavantaj olarak yine parity diski tek boğaz kalacak sistemi yavaşlatır.

RAID-5 (Dağıtılmış Eşlik): Parity'ler tek diske yığılmaz. Tüm disklere homojen dağılır. Günümüzde en sık kullanılan. Bottleneck'i önler.

RAID-6 (Çift Dağıtılmış Eşlik): İki katmanlı parity üretilir ve bunlar disklere dağıtılır.